populate it with content that can be queried. You can download this file from the following URL:

https://raw.githubusercontent.com/9dot9Media/fasttrack_to_cms_code_snippets/master/05_django_models/test_data_1.json

You can then import it into your database using the following command:

- `python3 manage.py loaddata test_data_1.json`

Django supports some very advanced data queries and manipulations that we wont go into in this text. We will however look at all the basic types of queries that are essential for nearly any application.

For our article list page, we will probably need to display a list of all articles. The way to ask the database for all Articles is:

- `Article.objects.all()`

Similarly if you want all Users / Authors:

- `User.objects.all()`

Actually, if we think about it, we don't want to display all articles. After all we have an keep the publish status for each article for a reason, we don't want articles not marked for publishing showing up. So we need some way to filter for only those Articles that have been published. This is easily accomplished with the following query:

- `Article.objects.filter(publish_status=True)`

Likewise we can provide 'publish_status=False' to get all unpublished articles. Another way to get a list of all articles that have their publish status set to true is to exclude all articles that have their publish status set to False, since it is a binary option. Here is how to do that:

- `Article.objects.exclude(publish_status=False)`

Let's find out how many of our articles start with 'The' shall we. This query can also be accomplished using the filter method:

- `Article.objects.filter(title__startswith='The')`

Simple right? Now let's make this a little more complex. How about we get a list of all the articles written by authors whose first name starts with a W? We don't know why you'd want to know this, and perhaps it isn't out business to know, but here's how you can get to that sweet data:

- `Article.objects.filter(author__first_name__startswith='v')`

Note here that the first name of the author isn't even stored on the same model, it is part of the User model which is stored on a different database table! Django just automatically manages this for you. Since we are looking up a field of model linked via the author field in the Article we need to use